به نام خدا

مهر ماه ۱۴۰۵

تمرین اول آزمون نرم افزار

استاد: دکتر احسان خامس‌پناه

«تست‌های پارامتری و JUnit»

سلام 👋

امیدواریم خوب باشید،

با توجه به روز افزون شدن مبحث استفاده از Ai 🤖، به احتمال زیاد کد تست‌های این تمرین توسط شما با Ai نوشته می‌شوند که کاملا درست است و هیچ اشکالی ندارد ! 👏👏

اما از آنجایی که مبحث تست نرم‌افزار چه در تحقیقات و چه در بازار کار آینده به کمک شما خواهد آمد، نیاز است تا بدانید Ai برای شما چه تستی و با استفاده از چه تکنولوژی‌ای نوشته است و فایده و تاثیر یک تست چیست تا در آینده Ai نتواند از دانش کم شما در این مبحث سواستفاده کند 🙃 و کد و تستی با کیفیت پایین به شما تحویل دهد. (امروزه یکی از راه های اعتبارسنجی کد های نوشته شده توسط Ai دقیقا مبحث تست نرم‌افزار و تست‌نویسی صحیح است.)

بنابراین این تمرین بر این اساس طراحی شده است که شما گام به گام فایل صورت تمرین را خودتان می خوانید 👓 و بخش هایی که نیازمند یادگیری هستند را یاد گرفته ✍️ و در بخش هایی که نیاز به کد نویسی وجود دارد از Ai 🤖 استفاده می‌کنید.

نحوه‌ی نمره‌دهی 🧱 و تحویل‌گیری 🫲 تمرین هم به این صورت خواهد بود: کد نوشته‌شده توسط شما و Ai بخش کوچکی از نمره‌ی تمرین را تشکیل خواهد داد و اکثر نمره در یک تحویل مجازی بر اساس سوالاتی که دستیاران از مفهوم تست‌های نوشته شده و نحوه ی کارکرد تست‌ها خواهند پرسید، مشخص می‌شود.

هدف این تمرین آشنایی با آزمون واحد (Unit Test) است که برای سنجش درستی عملکرد کوچک‌ترین بخش‌های برنامه به صورت جداگانه و مستقل از هم می‌باشد. جهت نوشتن تست‌ها از JUnit که framework زبان جاوا برای نوشتن آزمون نرم‌افزار و اجرای خودکار آن است استفاده می‌کنید. بعد از مطالعه این صورت تمرین اگر هر سوال و ابهامی بود می‌توانید از ما[1] بپرسید.

موفق باشید 💪

مهدی، محمد مهدی

---

[1] مهدی وجهی (mvajhi@ut.ac.ir) و محمد مهدی صمدی (t.me/M1M7S)

سلام بر جاوا ☕

با توجه به این که امروزه بسیاری از پروژه های بزرگ صنعتی با زبان جاوا توسعه داده می‌شوند و در این درس بخش عمده‌ای از تمرینات بر اساس این زبان جلو خواهند رفت، لازم است آشنایی مختصری با زبان جاوا و یکی از Dependency Manager و Build tool های محبوب آن یعنی Maven داشته باشید. در این رابطه لیستی از مباحثی که آشنایی با آن‌ها سبب می شود شما به راحتی تمرینات را پیش ببرید در زیر آمده است. می‌توانید از هر روشی که راحت هستید برای یادگیری استفاده کنید. اگر با کتاب ها سریع‌تر یاد می‌گیرید به سراغ کتاب بروید. اگر با ویدیوهای آموزش راحت‌تر هستید از آن‌ها استفاده کنید یا اگر علاقه دارید documentهای مربوط و اصلی را بخوانید! از آن‌ها کمک بگیرید و اگر با سوال و جواب و توضیح هوشواره (Ai) ها بهتر یاد می‌گیرید از آن ها استفاده کنید (بهترین گزینه!) فقط کافی است **اندکی** در مورد مطالب زیر بدانید (لازم نیست کاملا با تمام جزئیات بر آن‌ها مسلط شوید) و بتوانید از آن‌ها استفاده کنید که زمان زیادی نخواهد برد.

Java

- .java files, Java compiler, javac, Java bytecode and .class files
- How a Java program runs
- JVM, JDK, JRE and their relationships (what is Java 26 temurin)
- Basic Java Syntax: The structure of a Java program, The main method
- Variables and Data Types,
- null , final and static
- Inheritance: @override and extends
- Polymorphism, Abstract Classes and Interfaces
- Packages, Imports and Multi-package Projects
- Arrays, Lists, Enums
- Exceptions and try catch
- Java Annotations

Maven

- What Are Build Tools and Dependency Managers?
- What Is Maven?
- Why Do You Need Maven, and How Are Projects Managed Without It?
- Maven Project Structure: Files and Directories
- Where to Put Your Java Source Files and Test Files
- What Is the pom.xml File, and What Is XML?
- How to Add Dependencies and What Maven Repository is
- Essential Maven Commands for Building and Testing a Project

استفاده از IDE و محیط اجرای برنامه

بعد از این که با مفاهیم Java و Maven آشنا شدید آن‌ها را بسته به معماری ماشین و سیستم‌عامل خود نصب و راه‌اندازی کنید (لطفا از Java 26 temurin استفاده کنید). در ادامه می‌توانید از IDE مطلوب خود برای نوشتن و اجرای کدهای جاوا استفاده کنید اما یکی از IDEهای محبوب برای توسعه کدهای جاوا JetBrains IntelliJ است که نوشتن، تست و اجرای کد با استفاده از جاوا و Maven را ساده‌تر می‌کند. به

راحتی با کمک از هوشواره و اندکی search در اینترنت می‌توانید آموزش‌های نحوه‌ی راه‌اندازی، استفاده از maven و اجرای برنامه‌ی جاوا در Intellij را بیابید.

و اینک JUnit 🧪

در این تمرین شما با JUnit که framework مبتنی بر جاوا برای نوشتن کدهای تست خودکار کد نرم‌افزار است آشنا خواهید شد. تمرکز این تمرین استفاده از مبحث **Parameterized Tests** در Junit است. جهت آشنایی و آماده شدن برای استفاده از JUnit همانند آنچه در بخش سلام بر جاوا انجام شد نحوه‌ی استفاده از JUnit در یک پروژه‌ی جاوا و Maven به همراه موضوعات اسلاید هفتم درس را مرور کنید. حال که با زبان و فریم‌ورک مدنظر آشنا شدید، به سراغ بخش اصلی تمرین می‌رویم.
Code Base نرم‌افزار پروژه‌ی یک خودروی خودران که به زبان جاوا نوشته شده است در اختیار شما قرار گرفته است (مورد دوم بخش توضیحات را بخوانید) و از شما خواسته شده تا با توجه به حیاتی بودن تست خودروی خودران پیش از عرضه به بازار تست‌های قسمت نرم‌افزاری این پروژه را بر اساس کدی که در اختیار شما قرار گرفته است بنویسید. توجه کنید که تست‌های نوشته شده توسط شما از اهمیت بالایی برخوردار هستند و باید گستره‌ای از مفاهیم مهم را رعایت کنند (همانند آنچه در جلسه‌ی پنجم درس دیدید).

نوشتن تست

در این پروژه، classهایی که مورد تست قرار می گیرند در مسیر:
"src/main/java/autopilot/core"
قرار دارند. برای classهای:

- [ ] AdaptiveCruise
- [ ] BrakeController
- [ ] GearBox
- [ ] SteeringController

تست واحد (Unit Test) بنویسید. در نوشتن تست‌های خود به موارد زیر توجه کنید:

- تست‌های هر class را بر اساس نام class در فایل جداگانه و package متناظر با آن قرار دهید. برای مثال تست‌های مربوط به کلاس AutoDriver در فایل AutoDriverTest.java و مسیر "src/test/java/autopilot/core" قرار بگیرد.
- از این منبع یا منابع دیگر الگوهای مختلف نام‌گذاری methodهای تست را مطالعه و مقایسه کنید و یکی از آنان را در نام‌گذاری تمام تست‌های خود استفاده کنید.
- هدف، تعداد تست زیاد نیست. سعی کنید برای هر تابع به اندازه کافی بنویسید که تمام حالات ممکن آن آزموده شوند و تست تکراری ننویسید.
- تست‌ها را مستقل از هم طراحی کنید.

- نیازی به نوشتن تست برای methodهای constructor و getter/setter نیست.

بررسی تست

از قبل تست‌هایی برای کلاس AutoDriver نوشته شده‌اند. با بررسی آن‌ها، مواردی را که مطابق اصول تدریس شده نیستند شناسایی کرده و همین تست‌ها را ویرایش کنید یا از ابتدا بنویسید (نیازی نیست تست برای سناریوهای دیگر در این کلاس بنویسید). برای مثال هر یک از موارد زیر (یا هر ایراد دیگری) نشان از یک تست بد می‌دهد:

- نام‌گذاری نامفهوم تست،
- assertionهای بیش از حد در یک تست،
- وجود magic number و magic string،
- وجود کد یا assertion تکراری در دو یا چند تست که به جای آن می‌توان تستی برای بخش مشترک نوشت یا از تست پارامتری استفاده کرد.

تست پارامتری

بعضی اوقات نیاز داریم تستی را با چند ورودی مختلف اجرا کنیم. تست پارامتری از نوشتن کد تکراری برای هر ورودی جلوگیری می‌کند. جهت آشنایی با انواع مختلف ورودی آن، حتما تست‌هایی با sourceهای ورودی MethodSource، CsvSource، CsvFileSource، EnumSource و ValueSource بنویسید و هر کدام را برای سنجیدن method متفاوتی به کار ببرید.

توضیحات

- تمرین های درس در گروه های دو نفره انجام می شوند.
- Code Base این پروژه در این مخزن Github موجود است. کد ها را در یک مخزن خصوصی در Github بارگذاری کنید یا پروژه را fork کنید و کد تست‌های خود را به آن اضافه کنید.
- تا پایان تاریخ (۱۴۰۵/۰۸/۰۹) مخزن را خود را به صورت Private نگه دارید و پس از آن مخزن خود را به صورت Public دربیاورید تا دستیاران کد شما را بررسی کنند.
- در ایلرن آدرس مخزن Github تمرین خود را به همراه Hash آخرین کامیت (که تاریخ آن باید پیش از موعد تحویل باشد) را بارگذاری کنید.
- این تمرین برای آشنایی بیشتر و یادگیری عملی مطالب درس طراحی شده است. در صورت پیدا شدن تقلب، مطابق قوانین درس برخورد خواهد شد.